Estimating Peat Subsidence with BLUE

We start by assuming a **linear model** that describes the movement with an **offset and a linear trend**, and estimate these with Best Linear Unbiased Estimation (BLUE).

Data

We have two independent measurement techniques that observe the **same parcel**:

	InSAR	Extensometer
What is measured	Average height change of the **whole parcel** (radar satellite)	Height change at **one point** in the parcel (instrument in a borehole)
Sampling	every 6 days (some epochs missing due to lack of coherence)	daily
Precision	$\sigma_{\text{InSAR}} = 3$ mm	$\sigma_{\text{ext}} = 8$ mm

For both datasets you may assume that the observations are **normally distributed and independent**, with the standard deviations given in the table.

In addition, the file `weather.csv` contains daily precipitation and evapotranspiration for the area. We will only use it to help interpret the results this week; next week it becomes part of the model.

All files are in the folder `data/`. Every file has a column `Day`, which is the number of days since 1 January 2019 (the first day of the weather record, which starts one year before the observations). You can use it to link observations to the weather.

In [87]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.stats.distributions import chi2

np.set_printoptions(precision=3)
```

Part 1

Download and read the data

In [88]:
```
import os
from urllib.request import urlretrieve

def findfile(fname):
    if not os.path.isfile(fname):
        print(f'Downloading {fname}...')
```

```
        urlretrieve('https://github.com/TUDelft-MUDE/source-files/raw/mai

findfile('insar_observations.csv')
findfile('extensometer_observations.csv')
findfile('weather.csv')
```

In [89]:
```
insar = pd.read_csv('insar_observations.csv')
ext = pd.read_csv('extensometer_observations.csv')
weather = pd.read_csv('weather.csv')

dates_insar = pd.to_datetime(insar['Dates'])
day_insar = insar['Day'].to_numpy()
y_insar = insar['Observations[m]'].to_numpy() * 1000      # mm

dates_ext = pd.to_datetime(ext['Dates'])
day_ext = ext['Day'].to_numpy()
y_ext = ext['Observations[m]'].to_numpy() * 1000          # mm

dates_weather = pd.to_datetime(weather['Dates'])
P = weather['Precipitation[mm]'].to_numpy()                # mm/day
E = weather['Evapotranspiration[mm]'].to_numpy()           # mm/day


sigma_insar = 3   # mm
sigma_ext = 8     # mm
```

> **Task 1.1:**
>
> Investigate the two displacement datasets: print some summary statistics (number of observations, first and last date, minimum, maximum, ...) and plot both time series in one figure. Also make a plot of the precipitation (P) and evapotranspiration (E).
>
> Describe the two datasets in your own words. Consider at least:
>
> 1. the differences in sampling and precision;
> 2. what each technique physically observes (what could be the reason the plots of the two measurements are slightly different);
> 3. the patterns you see in the time series.
>
> *Note down your findings*

In [90]:
```
# Print summary statistics of both displacement datasets
print('summary statistics of InSAR dataset:', len(insar), insar.describe(
print('summary statistics of extensometer dataset:', len(ext), ext.descri

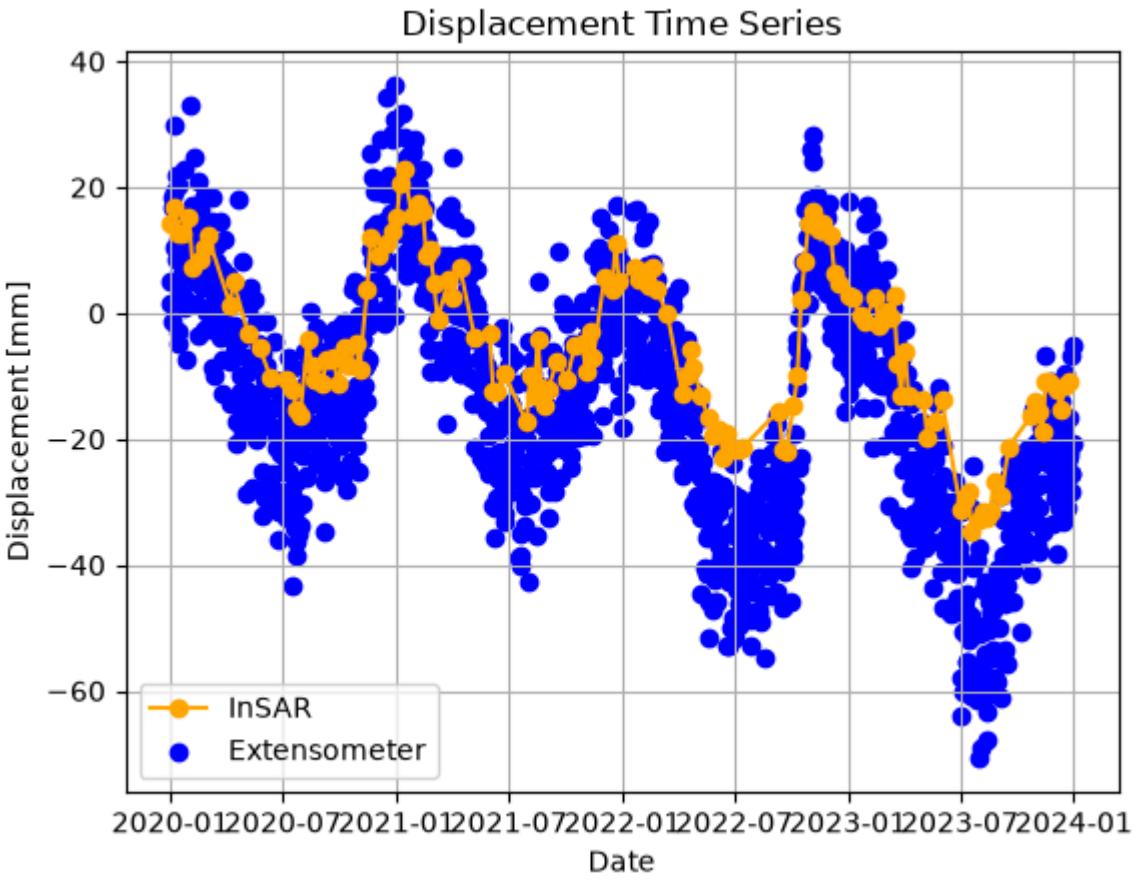
# Plot both time series in one figure, and the weather data in a second f
### YOUR CODE LINES HERE

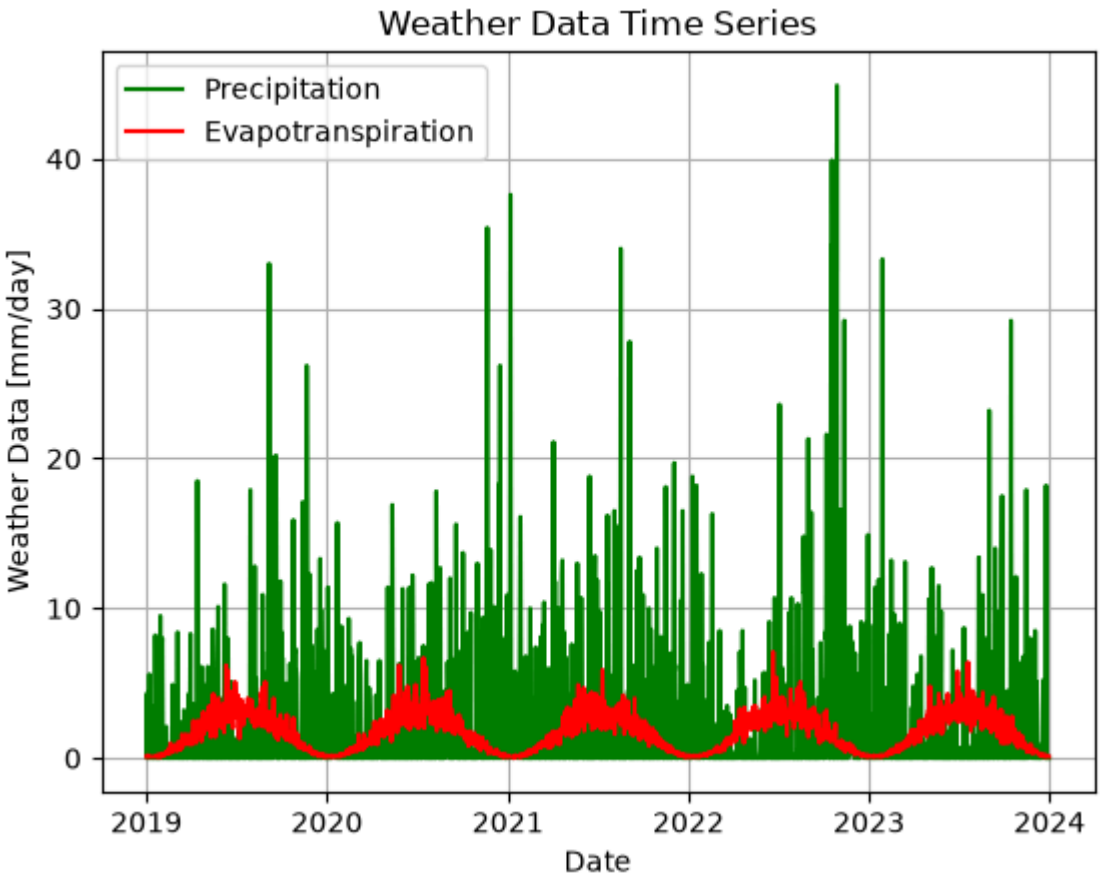
plt.figure()
plt.plot(dates_insar, y_insar, 'o-', label='InSAR', color='orange')
plt.scatter(dates_ext, y_ext, label='Extensometer', color='blue')
plt.xlabel('Date')
plt.ylabel('Displacement [mm]')
plt.title('Displacement Time Series')
```

```
plt.legend()
plt.grid()
plt.show()

plt.figure()
plt.plot(dates_weather, P, label='Precipitation', color='green')
plt.plot(dates_weather, E, label='Evapotranspiration', color='red')
plt.xlabel('Date')
plt.ylabel('Weather Data [mm/day]')
plt.title('Weather Data Time Series')
plt.legend()
plt.grid()
plt.show()
```

```
summary statistics of InSAR dataset: 145                Day  Observations
[m]
count   145.000000       145.000000
mean   1110.696552        -0.004959
std     426.242128         0.013127
min     365.000000        -0.034700
25%     737.000000        -0.013230
50%    1121.000000        -0.005940
75%    1481.000000         0.005280
max    1817.000000         0.022940 first date: 365 last date: 1817
summary statistics of extensometer dataset: 1461                Day  Obser
vations[m]
count  1461.000000      1461.000000
mean   1095.000000        -0.013515
std     421.898685         0.019199
min     365.000000        -0.070700
25%     730.000000        -0.027370
50%    1095.000000        -0.012700
75%    1460.000000         0.001400
max    1825.000000         0.036190 first date: 365 last date: 1825
```





Part 2: Functional model

We start with the simplest model for the height change d at time t: an offset and a constant velocity,

$$ d(t) = d_0 + v\, t, $$

where d_0 [mm] is the relative surface height at $t=0$ and v [mm/year] is the velocity. We define t in **years since 1 January 2020** (the first observation epoch), so that d_0 refers to the same moment for both datasets.

We fit this model **separately** to the InSAR data and to the extensometer data.

> **Task 2.1:**
>
> Compute the time t in years for both datasets and set up the design matrices `A_insar` and `A_ext`.
>
> *Hint:* the column `Day` is the number of days since 1 January 2019; 1 January 2020(first day of extensometer and InSAR observations) is day 365. Use 365.25 days per year.

In [91]:
```
day_0 = 365
t_insar = (day_insar - day_0) / 365.25
t_ext = (day_ext - day_0) / 365.25


# A_insar = []
# for i in range(len(t_insar)):
#     A_insar.append([1, t_insar[i]])


# A_ext = []
# for i in range(len(t_ext)):
#     A_ext.append([1, t_ext[i]])
A_insar = np.array([[1, t] for t in t_insar])
A_ext   = np.array([[1, t] for t in t_ext])
print('First 3 rows of A_insar:\n', A_insar[:3])
print('First 3 rows of A_ext:\n', A_ext[:3])
```

```
First 3 rows of A_insar:
 [[1.    0.   ]
 [1.    0.016]
 [1.    0.033]]
First 3 rows of A_ext:
 [[1.    0.   ]
 [1.    0.003]
 [1.    0.005]]
```

> **Task 2.2:**
>
> Answer the following questions (use the code cell to compute the numbers):
>
> 1. Write down the unknown parameters of the functional model, and their units
> 2. How many observations m and unknowns n are there for each dataset?

Make sure to note down your findings with pen and paper

In [92]:
```
### YOUR CODE LINES HERE
print('the number of unkowns in the model is 2, the intercept [m] and the
print(' the number of observations in the insar dataset is', len(y_insar)
print(' the number of observations in the extensometer dataset is', len(y
```

```
the number of unkowns in the model is 2, the intercept [m] and the slope o
f the linear trend [m/year]
 the number of observations in the insar dataset is 145  and the number of
unknowns is 2
 the number of observations in the extensometer dataset is 1461  and the n
umber of unknowns is 2
```

Part 3: Stochastic model

The stochastic model describes the precision of the observations with the covariance matrix Σ_Y.

> **Task 3.1:**
>
> Set up the covariance matrices `Sigma_Y_insar` and `Sigma_Y_ext`.
>
> Print the shapes of the covariance matrices
>
> Print the first 3x3 entries

In [93]:
```
Sigma_Y_insar = np.diag([sigma_insar**2] * len(y_insar))
Sigma_Y_ext = np.diag([sigma_ext**2] * len(y_ext))

print('shape of Sigma_Y_insar:', Sigma_Y_insar.shape)
print('shape of Sigma_Y_ext:', Sigma_Y_ext.shape)
print('Sigma_Y_insar (first 3x3):\n', Sigma_Y_insar[:3, :3])
print('Sigma_Y_ext (first 3x3):\n', Sigma_Y_ext[:3, :3])
```

```
shape of Sigma_Y_insar: (145, 145)
shape of Sigma_Y_ext: (1461, 1461)
Sigma_Y_insar (first 3x3):
 [[9 0 0]
 [0 9 0]
 [0 0 9]]
Sigma_Y_ext (first 3x3):
 [[64  0  0]
 [ 0 64  0]
 [ 0  0 64]]
```

> **Task 3.2:**
>
> 1. What is the dimension of both Σ_Y matrices?
> 2. What value is on the diagonal for both Σ_Y matrices?

note down your findings with pen and paper

Part 4: Apply Best Linear Unbiased Estimation

> **Task 4.1:**
>
> Complete the function `BLUE` below and use it to estimate the unknown parameters and the parameter covariance matrix for both datasets.
>
> 1. print the unknown parameters
> 2. print the covariance matrix of the parameters
>
> *note down the your computations and outcome on paper*

In [94]:
```
def BLUE(A, y, Sigma_Y):
    '''Best Linear Unbiased Estimator.

    Input:  A, y , Sigma_Y
    Output: xhat, Sigma_X
    '''
    xhat = np.linalg.inv(A.T @ np.linalg.inv(Sigma_Y) @ A) @ A.T @ np.lin
    Sigma_X = np.linalg.inv(A.T @ np.linalg.inv(Sigma_Y) @ A)
    return xhat, Sigma_X

xhat_insar, Sigma_X_insar = BLUE(A_insar, y_insar, Sigma_Y_insar)
xhat_ext, Sigma_X_ext = BLUE(A_ext, y_ext, Sigma_Y_ext)

print('Estimated parameters for InSAR dataset:', xhat_insar, 'with covari
print('Estimated parameters for Extensometer dataset:', xhat_ext, 'with c
```

```
Estimated parameters for InSAR dataset: [ 6.003 -5.369] with covariance ma
trix:
 [[ 0.253 -0.094]
 [-0.094  0.046]]
Estimated parameters for Extensometer dataset: [ 2.368 -7.947] with covari
ance matrix:
 [[ 0.175 -0.066]
 [-0.066  0.033]]
```

Part 5: Model predictions and residuals

Compute the estimated observations $\hat{Y}$ and the residuals $\hat{\epsilon}$.

> **Task 5.1:**
>
> Complete the function `get_results` to compute $\hat{Y}$, $\hat{\epsilon}$ and the standard deviations of both.

In [95]:
```
def get_results(A, y, Sigma_Y, xhat, Sigma_X):

    yhat = A @ xhat
    ehat = y - yhat
    sigma_yhat = A @ np.linalg.inv(A.T @ np.linalg.inv(Sigma_Y) @ A) @ A.
```

```
    std_yhat = np.sqrt(np.diag(sigma_yhat))
    sigma_ehat = Sigma_Y - sigma_yhat
    std_ehat = np.sqrt(np.diag(sigma_ehat))
    return yhat, ehat, std_yhat, std_ehat

yhat_insar, ehat_insar, std_yhat_insar, std_ehat_insar = get_results(A_in
yhat_ext, ehat_ext, std_yhat_ext, std_ehat_ext = get_results(A_ext, y_ext

print('First 5 predicted values for InSAR dataset:', yhat_insar[:5])
print('First 5 predicted values for Extensometer dataset:', yhat_ext[:5])
print('First 5 residuals for InSAR dataset:', ehat_insar[:5])
print('First 5 residuals for Extensometer dataset:', ehat_ext[:5])
print('First 5 standard deviations of predicted values for InSAR dataset:
print('First 5 standard deviations of predicted values for Extensometer d
print('First 5 standard deviations of residuals for InSAR dataset:', std_
print('First 5 standard deviations of residuals for Extensometer dataset:
```

```
First 5 predicted values for InSAR dataset: [6.003 5.915 5.827 5.739 5.56
2]
First 5 predicted values for Extensometer dataset: [2.368 2.347 2.325 2.30
3 2.281]
First 5 residuals for InSAR dataset: [ 8.267 10.885  6.933  6.921  9.538]
First 5 residuals for Extensometer dataset: [-0.708  2.583 -3.725 16.157 1
4.529]
First 5 standard deviations of predicted values for InSAR dataset: [0.503
0.5   0.497 0.494 0.488]
First 5 standard deviations of predicted values for Extensometer dataset:
[0.418 0.418 0.418 0.417 0.417]
First 5 standard deviations of residuals for InSAR dataset: [2.957 2.958
2.959 2.959 2.96 ]
First 5 standard deviations of residuals for Extensometer dataset: [7.989
7.989 7.989 7.989 7.989]
```

> **Task 5.2:**
>
> Complete the function `plot_results` below by computing the 98% confidence intervals of $\hat{Y}$ and $\hat{\epsilon}$.

In [96]:
```
def plot_results(name, dates, y, yhat, ehat, std_yhat, std_ehat, sigma, a
    '''Plot observations + fitted model, residuals and a histogram of the
    k98 = norm.ppf(1 - alpha/2)
    CI_y = k98 * np.sqrt(np.diag(y))
    CI_e = k98 * std_ehat  # Assuming the same for residuals
    CI_yhat = k98 * std_yhat  # Assuming the same for predicted values
    CI_label = f'{100 * (1 - alpha):.0f}% confidence region'
    fig = plt.figure(figsize=(14, 8))
    ax1 = plt.subplot2grid((2, 3), (0, 0), colspan=3)
    ax2 = plt.subplot2grid((2, 3), (1, 0), colspan=2)
    ax3 = plt.subplot2grid((2, 3), (1, 2))

    ax1.plot(dates, y, '.', color='k', ms=3, label='observations')
    ax1.plot(dates, yhat, color='tab:red', label='fitted model')
    ax1.fill_between(dates, yhat - CI_yhat, yhat + CI_yhat,
                     color='tab:red', alpha=0.3, label=f'CI yhat {100 * (

    ax1.set_xlim(dates.iloc[0], dates.iloc[-1])
    ax1.set_ylabel('displacement [mm]')
    ax1.set_title(f'{name}: observations and fitted model')
```

```
    ax1.legend(loc='lower left')

    ax2.plot(dates, ehat, '.', color='k', ms=3)
    ax2.fill_between(dates, -CI_e, CI_e, color='tab:red', alpha=0.2,
                     label=f'CI ehat {100 * (1 - alpha):.0f}%')

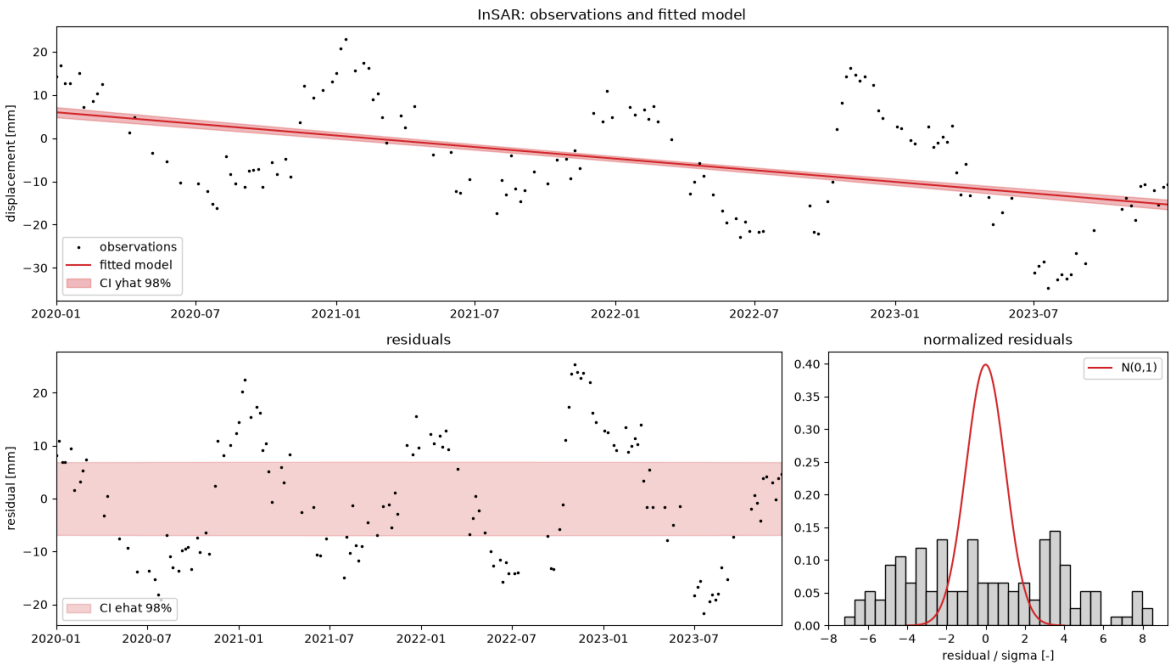
    ax2.set_xlim(dates.iloc[0], dates.iloc[-1])
    ax2.set_ylabel('residual [mm]')
    ax2.set_title('residuals')
    ax2.legend(loc='lower left')

    ax3.hist(ehat / sigma, bins=30, density=True, color='lightgrey', edge
    xx = np.linspace(-4, 4, 200)
    ax3.plot(xx, norm.pdf(xx), color='tab:red', label='N(0,1)')
    ax3.set_xlabel('residual / sigma [-]')
    ax3.set_title('normalized residuals')
    ax3.legend()
    plt.tight_layout()
    plt.show()

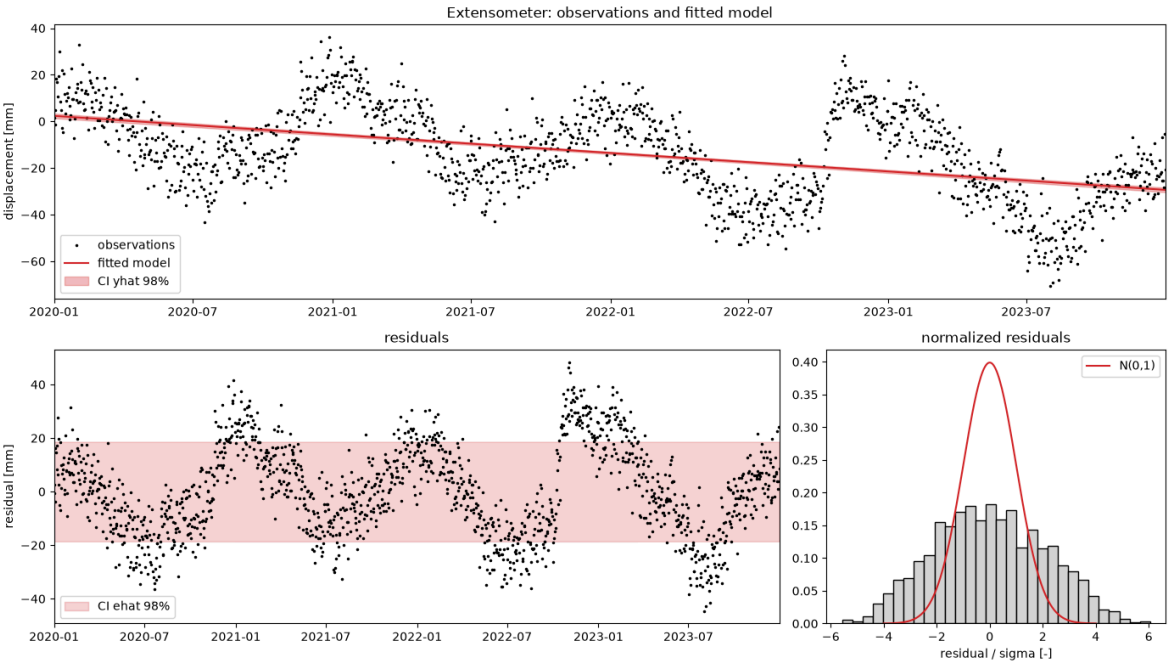
    print(f'{name}: mean residual = {ehat.mean():.2f} mm, std residual =
          f'(sigma = {sigma} mm)')
```

In [97]:
```
plot_results('InSAR', dates_insar, y_insar, yhat_insar, ehat_insar,
             std_yhat_insar, std_ehat_insar, sigma_insar, alpha=0.02)
plot_results('Extensometer', dates_ext, y_ext, yhat_ext, ehat_ext,
             std_yhat_ext, std_ehat_ext, sigma_ext, alpha=0.02)
```

```
C:\Users\thijm\AppData\Local\Temp\ipykernel_42136\3516165911.py:4: Runtime
Warning: invalid value encountered in sqrt
  CI_y = k98 * np.sqrt(np.diag(y))
```




```
InSAR: mean residual = 0.00 mm, std residual = 11.50 mm (sigma = 3 mm)
```




```
Extensometer: mean residual = -0.00 mm, std residual = 16.86 mm (sigma = 8 mm)
```

> **Task 5.3:**
>
> Write down your observations for each figure: the graphs of $\hat{y}$ and $\hat{e}$ and the distribution of the normalized residuals.

Part 6: A model driven by the weather (Model 2)

In this part we build a model that uses the weather data to explain the surface movement, while keeping it **linear**, so that we can still use BLUE.

Lets check out the weather data (we loaded this already in the beginning of the notebook)

What is in `weather.csv`?

The file contains one row for **every day** from 1 January 2019 to 31 December 2023. The weather starts one year *before* the first observation (1 January 2020). We explain why below.

Column	Unit	Meaning
`Dates`	–	calendar date
`Day`	days	number of days since 1 January 2019 (day 0)
`Precipitation[mm]`	mm/day	precipitation P: water that comes **in** (rain)
`Evapotranspiration[mm]`	mm/day	evapotranspiration E: water that goes **out** (evaporation from the soil + transpiration by the grass)

Column	Unit	Meaning
`Dry[-]`	–	1 if the day is **dry**, 0 if it is wet (definition below)
`DryDays[days]`	days	$D(t)$: the **number of dry days** counted from the start of the file up to and including that day

The difference $P - E$ [mm/day] is the **net water input** of the parcel. Positive means the parcel gets wetter that day; negative means it dries out.

`Day` shows the number of days since 1 January 2019 (starts at 0). So for an observation on day `Day`, the weather on that day is simply `P[Day]`, `D[Day]`, etc.

Two processes, two weather variables

Lets introduce two processes that we think will affect the movement of peat ground. We describe each of them with a variable computed from the weather file.

1. Irreversible subsidence: the dry-day counter $D(t)$

Peat only oxidises when it is **dry**, so the ground only goes down on dry days. A day is **dry** when the net water input over the last few days (including that day) is zero or negative.

$D(t)$ is the number of dry days up to day t. It goes up by 1 on a dry day and stays the same on a wet day. You can find it in the column `DryDays` of the weather file.

The irreversible subsidence is then:

$$x_I \, D(t)$$

where x_I is the subsidence per dry day [mm/day]. This replaces the $v\,t$ term of the linear trend.

2. Reversible motion: the water surplus $S_\tau(t)$

The soil swells when it gets wetter and shrinks when it dries out. How wet it is depends on today's weather and also on the days and weeks before: the soil has a *memory*. We model this as a **leaky bucket**. Each day the net input $P - E$ is added, and part of the old content leaks away:

$$ S_i = e^{-1/\tau}\, S_{i-1} + (P_i - E_i) $$

S_i is the **water surplus** [mm] on day i. Because each day depends on the day before, you compute it day by day. Start with an empty bucket on the first day of the weather record. We write it as $S_\tau(t)$ because it depends on τ.

τ [days] is the soil's memory. Each day the bucket keeps a fraction $e^{-1/\tau}$ of its content. For example, $\tau = 10$ gives $e^{-1/10} \approx 0.90$, so about 10% leaks away each day.

- Small τ: the soil reacts fast and forgets fast.
- Large τ: the soil reacts slowly and remembers longer.

The reversible motion is then:

$$k \, S_\tau(t)$$

where k [-] is a scaling factor.

Why does the weather data start one year earlier? We start the bucket empty on 1 January 2019, which is not the real state of the soil. After a year, the effect of that starting value is negligible. The dry-day counter also needs the previous days (we computed this already for you in `DryDays`). This first year is called the *spin-up* period.

The new functional model

Combining both processes gives

$$ d(t) = d_0 + x_I \, D(t) + k \, S_\tau(t) . $$

In this assignment we **fix** $\tau = 10$ days. Then $D(t)$ and $S_{10}(t)$ are known numbers for every day, and the three unknowns d_0, x_I and k.

> **Task 6.1:**
>
> 1. Load the dry-day counter D from the column `DryDays[days]` of the weather file.
> 2. Complete the function `water_surplus` below and compute the water surplus S with $\tau = 10$ days for every day of the weather record.
> 3. Plot both $D(t)$ and $S_{10}(t)$ for every day of the weather record
>
> Then run the cell after it to plot S and D.

In [98]:
```
weather.head()
print(weather.columns)
```

```
Index(['Dates', 'Day', 'Precipitation[mm]', 'Evapotranspiration[mm]', 'Dry
[-]',
       'DryDays[days]'],
      dtype='str')
```

In [99]:
```
D = weather['DryDays[days]']

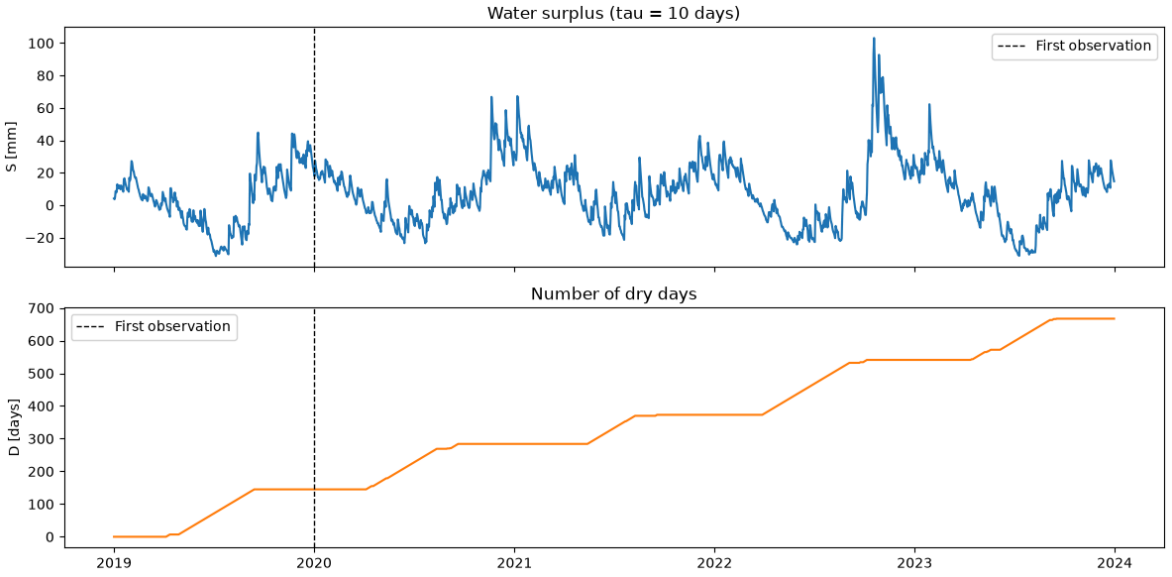
def water_surplus(P, E, tau):
    '''Leaky-bucket water surplus S [mm] for every day of the weather rec
```

```
    a = np.exp(-1 / tau)
    S = np.zeros(len(P))
    S_previous = 0.0
    for i in range(len(P)):
        S[i] = a * S_previous + P[i] - E[i]
        S_previous = S[i]
    return S

tau = 10   # days (fixed)
S = water_surplus(P, E, tau)
```

In [100…

```
fig, ax = plt.subplots(2, 1, figsize=(12, 6), sharex=True)
ax[0].plot(dates_weather, S)
ax[0].set_ylabel('S [mm]')
ax[0].set_title(f'Water surplus (tau = {tau} days)')
ax[1].plot(dates_weather, D, color='tab:orange')
ax[1].set_ylabel('D [days]')
ax[1].set_title('Number of dry days')
for a in ax:
    a.axvline(dates_insar.iloc[0], color='k', ls='--', lw=1, label='First
for a in ax:
    a.legend()
plt.tight_layout()
```



> **Task 6.2:**
>
> Describe the behavior of S and D over the year. (make a sketch of the plots)

> **Task 6.3:**
>
> Set up the design matrices `A2_insar` and `A2_ext` for the new model and estimate the unknowns with your `BLUE` function. Use the column `Day` (same as you did for your design matrix for model 1) to pick the values of D and S on the observation days.

In [101…
```
A2_insar = []
for i in range(len(t_insar)):
    A2_insar.append([1, t_insar[i], S[day_insar[i]]])

A2_insar = np.array(A2_insar)
A2_ext = []
for i in range(len(t_ext)):
    A2_ext.append([1, t_ext[i], S[day_ext[i]]])

A2_ext = np.array(A2_ext)
xhat2_insar, Sigma_X2_insar = BLUE(A2_insar, y_insar, Sigma_Y_insar)
xhat2_ext, Sigma_X2_ext = BLUE(A2_ext, y_ext, Sigma_Y_ext)
```

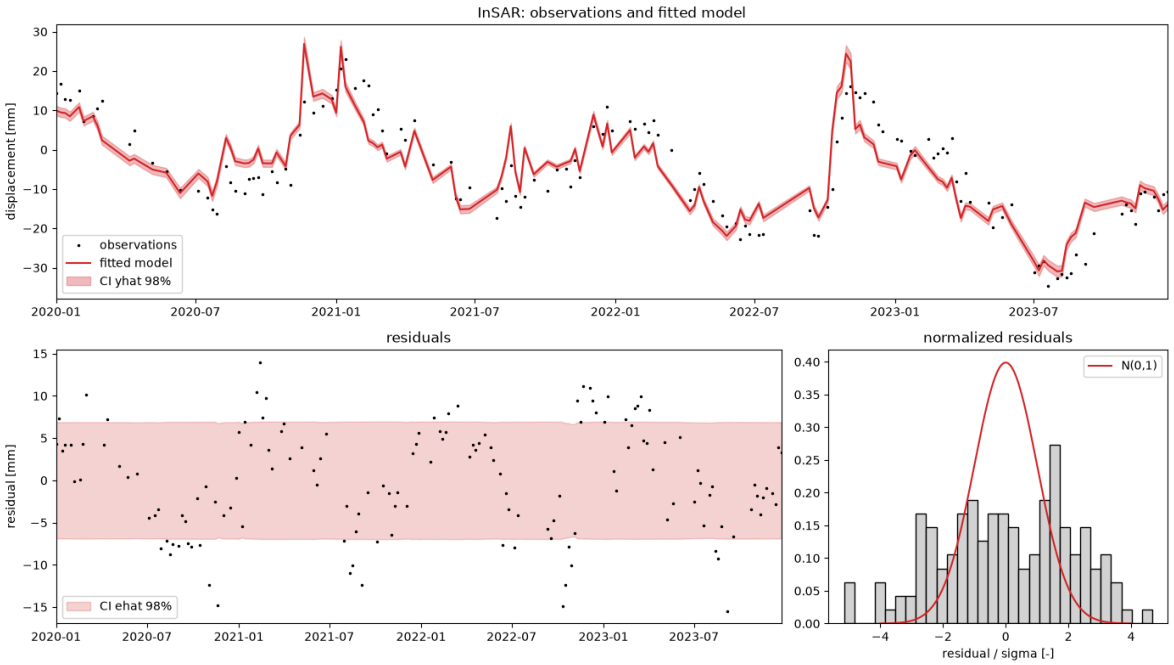
> **Task 6.4:**
>
> Compute the adjusted observations and residuals of the new model by calling your `get_results` function, plot them with `plot_results` (plot again the 98% confidence level)

In [102…
```
yhat2_insar, ehat2_insar, std_yhat2_insar, std_ehat2_insar = get_results(
yhat2_ext, ehat2_ext, std_yhat2_ext, std_ehat2_ext = get_results(A2_ext,

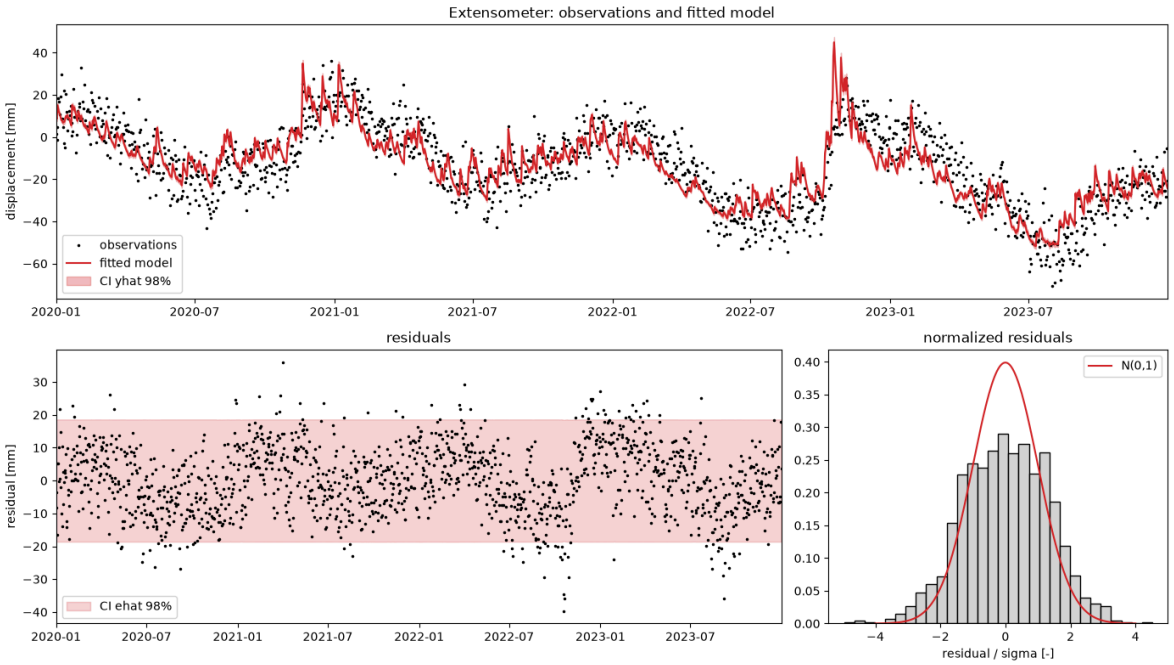
print('First 5 predicted values for InSAR dataset:', yhat2_insar[:5])
print('First 5 predicted values for Extensometer dataset:', yhat2_ext[:5]
print('First 5 residuals for InSAR dataset:', ehat2_insar[:5])
print('First 5 residuals for Extensometer dataset:', ehat2_ext[:5])
print('First 5 standard deviations of predicted values for InSAR dataset:
print('First 5 standard deviations of predicted values for Extensometer d
print('First 5 standard deviations of residuals for InSAR dataset:', std_
print('First 5 standard deviations of residuals for Extensometer dataset:

plot_results('InSAR', dates_insar, y_insar, yhat2_insar, ehat2_insar,
             std_yhat2_insar, std_ehat2_insar, sigma_insar, alpha=0.02)
plot_results('Extensometer', dates_ext, y_ext, yhat2_ext, ehat2_ext,
             std_yhat2_ext, std_ehat2_ext, sigma_ext, alpha=0.02)
```

```
First 5 predicted values for InSAR dataset: [ 9.947  9.488  9.299  8.475 1
0.843]
First 5 predicted values for Extensometer dataset: [ 9.983  8.609 15.097 1
3.233 11.544]
First 5 residuals for InSAR dataset: [4.323 7.312 3.461 4.185 4.257]
First 5 residuals for Extensometer dataset: [ -8.323  -3.679 -16.497   5.2
27   5.266]
First 5 standard deviations of predicted values for InSAR dataset: [0.514
0.509 0.505 0.499 0.507]
First 5 standard deviations of predicted values for Extensometer dataset:
[0.435 0.43  0.464 0.452 0.442]
First 5 standard deviations of residuals for InSAR dataset: [2.956 2.957
2.957 2.958 2.957]
First 5 standard deviations of residuals for Extensometer dataset: [7.988
7.988 7.987 7.987 7.988]
C:\Users\thijm\AppData\Local\Temp\ipykernel_42136\3516165911.py:4: Runtime
Warning: invalid value encountered in sqrt
  CI_y = k98 * np.sqrt(np.diag(y))
```




```
InSAR: mean residual = 0.00 mm, std residual = 6.30 mm (sigma = 3 mm)
```




```
Extensometer: mean residual = 0.00 mm, std residual = 10.47 mm (sigma = 8
mm)
```

> **Task 6.5:**
>
> 1. Interpret the estimated parameters. Convert x_I to mm/year (compute how many days per year on average the parcel has) and compare it with the velocity v from Part 4 (model 1).
> 2. Compare the estimated observations and residuals with those of model 1, compare the figures.
>
> *Note down your findings with pen and paper*

In [103…

```
# average number of dry days per year during the observation period (2020
first_day, last_day = dates_weather.iloc[0], dates_weather.iloc[-1]
years = (last_day - first_day).days / 365.25
```

```
N_dry = (weather['DryDays[days]'] == 1).sum()  / years
print(f'average number of dry days per year: {N_dry*365.25:.1f}')

#convert x_I from mm/day to v_I mm/year to compare with velocity estimate
v_I = xhat2_insar[2] * N_dry
print(f'Estimated velocity due to water surplus: {v_I:.2f} mm/year')
```

```
average number of dry days per year: 73.1
Estimated velocity due to water surplus: 0.09 mm/year
```

Do a double check and make sure you wrote all your findings down with pen and paper, so that you can discuss or refer to them when answering the questions in the report

> By `Isabel Slingerland`, Delft University of Technology. CC BY 4.0, more info on the Credits page of Workbook.